CS536 Final Project

Author

1 Motivation

Throughout this course, we have mostly considered learning settings where there is a single entity handling training and testing. However, these techniques are not always applicable to scenarios like edge computing and IoT devices. Often, there may be multiple devices with disjoint data sets that need to be aggregated to train a shared model. These devices may also have bandwidth or privacy requirements, preventing users from directly aggregating data then training. This is addressed by techniques in federated learning [7].

There are several ways data can be split among multiple devices. In the first case, each device can hold a subset of the data, recording different events with identical features. This is the problem that horizontal federated learning solves [4]. Alternatively, perhaps each device records some information about the same event, but they hold a disjoint set of features. We address this problem with vertical federated learning [2].

2 Horizontal Federated Learning

Let's begin by formalizing the horizontal federated learning problem. Suppose we have n different sensors. Our data will be $\mathbf{x} \in \mathbb{R}^{k \times m}$ where k is the number of distinct data points and m is the number of features associated with a labels $y \in \mathbb{R}^k$. Our goal is to either find a global model $f : \mathbb{R}^m \to \mathbb{R}$ that predicts the label y given $\mathbf{x} \in \mathbb{R}^m$ or, to have access to local models that can do the same. A 'global' model is constant across sensors while a 'local' model applies to a specific sensor. The data may not be IID, and so it is possible that each sensor is tasked with discovering a unique underlying distribution.

The task is called 'horizontal' because each sensor will be given a subset of the k data points. For our purposes, we will be assuming each sensor gets the same number of data points to train on.

2.1 Data

There are two distinct data types we explore: IID and heterogeneous. For the IID case, we generate n samples based off *events* number of features. Only *output_arity* number of features influence the label, the rest are noise. The input features $\mathbf{x}$ are uniformly distributed between -10 and 10. The label y

is generated by choosing a random subset of *output_arity* features and taking a linear combination of them with their weights being determined according to $\mathcal{N}(0, 1)$. This is then split across the sensors equally.

For the heterogeneous data, we generate the input features in the same way. For the labels, we again choose a subset of features. We then generate *base_weight* for each feature according to a standard normal distribution. Then, for each sensor, we have a $local\_weight \sim \mathcal{N}(base\_weight, heterogeneity\_std)$. This means each sensor has its own parameters for its underlying distribution.

2.2 Algorithms

We will explore three algorithms for federated learning explored in [1]. FedAvg works to construct a global model and is thus applicable to IID data. Our implementation of FedAvg:

1. We initialize weights and intercepts for a linear regression model.
2. The central server will broadcast the global model.
3. Each sensor runs stochastic gradient descent and returns the new local weights.
4. Take the mean of all the local weights, this becomes the new global model.
5. repeat steps 2 - 5 for some number of iterations.

Model Agnostic Meta Learning (MAML) [3] aims to find a global model that can easily be adjusted (via gradient descent) to a local learning task. The way to evaluate this algorithm is given the global model, run some number of local update steps with local validation data. Then, evaluate the MSE of that model against testing data. Our implementation of MAML:

1. Initialize weights and intercepts for a linear regression model. (In our code, you will see a single layer NN)
2. Broadcast the global model weights θ.
3. Each sensor runs stochastic gradient descent and gets new hypothetical weights θ' which are a function of θ.
4. Take the gradient of the sum of the mean squared error across all sensors with respect to θ and update the global model with a gradient descent step.
5. repeat steps 2 - 4 for some number of iterations.

Note that we are taking the gradient of a gradient here to find the optimal meta parameter θ.

Federated Multi Task Learning (FTML) aims to find a unique model for each sensor with the assumption that they are working on different but related learning tasks. We implemented the MOCHA algorithm [5]

$$TODO : explination \quad of \quad mocha$$

2.3 Questions of Interest

1. FedAvg generates one global model, so we expect it to perform poorly on IID data.
2. FMTL and MAML are different strategies for tackling heterogeneous distributed data, under what conditions does FTML outperform MAML and vice versa?
3. How does the training dataset size affect performance?
4. How does the number of sensors affect performance?
5. How does the output arity affect performance?
6. How does the number of adaptation steps in MAML affect the test MSE?
7. What is the rate of convergence for these models?

3 Vertical Federated Learning

In a general learning problem, we have $\mathbf{x} \in \mathbb{R}^m$ as an input vector and we want to learn some function $f : \mathbb{R}^m \to \mathbb{R}$ and predict the value $y = f(\mathbf{x})$. Consider the k party case, where we have $m = \sum_{i=1}^{k} m_i$, and $\mathbf{x} = \bigoplus_{i=1}^{m} \mathbf{x}_i \in \bigoplus_{i=1}^{m} \mathbb{R}^{m_i}$. Party i can only see $\mathbf{x}_i, y$, and needs to cooperate with other parties to predict $f(\mathbf{x})$ without divulging their own data. We implement a simplified version of [6], as there is a data leakage attack which the extra random vector in said paper's implementation does not directly address.

For simplicity, consider the case of linear regression. Take $f(\mathbf{x}) = \mathbf{x} \cdot \mathbf{w} = \sum_{i=1}^{k} \mathbf{x}_i \cdot \mathbf{w}_i$ for $\mathbf{w} \in \mathbb{R}^m$, where $\mathbf{w} = \sum_{i=1}^{k} \mathbf{w}_i \in \bigoplus_{i=1}^{m} \mathbb{R}^{m_i}$ and party i only sees $\mathbf{w}_i$. Given some $\mathbf{x}_i$, y, party i will compute and broadcast $\mathbf{x}_i \cdot \mathbf{w}_i$, then collect $\hat{y} = \sum_{i=1}^{k} \mathbf{x}_i \cdot \mathbf{w}_i$ from other parties. The goal is to minimize the squared error $\mathcal{L} = (y - \hat{y})^2$.

To train this model, some data $\mathbf{X} = (\mathbf{x}^1, \ldots, \mathbf{x}^N)^\intercal \in \mathbb{R}^{N \times m}$ and output $\mathbf{y}(y_1, \ldots, y_N)^\intercal \in \mathbb{R}^N$ is partitioned to $\mathbf{X}_i \in \mathbb{R}^{N \times m_1}$. Party i obtains $\mathbf{X}_i$, $\mathbf{y}$, and initializes $\mathbf{w}_i$ to a random vector. Then we run several rounds of gradient descent, adjusting w_l for each feature $l \in [m]$ as follows.

$$\frac{\partial \mathcal{L}}{\partial w_l} = \frac{\partial}{\partial w_l} \sum_{i=1}^{N} \left(\sum_{j=1}^{m} \mathbf{x}_i(j) \cdot w_j - y_i \right)^2 \tag{1}$$

$$= \sum_{i=1}^{N} 2\mathbf{x}_i(l) \left(\sum_{j=1}^{m} \mathbf{x}_i(j) \cdot w_j - y_i \right) \tag{2}$$

$$= \sum_{i=1}^{N} 2\mathbf{x}_i(l)(\mathbf{x}_i \cdot \mathbf{w} - y_i) \tag{3}$$

The key insight here is that the $\mathbf{x}_i \cdot \mathbf{w} - y_i$ can be computed without view of each individual feature, thus updating w_l only requires $\mathbf{x}_i(l)$. The party that needs to update w_l also owns $\mathbf{x}_i(l)$, so federated gradient descent is possible.

3.1 Experiments

We randomly generate 1000 data points with $\mathbf{x} \in \mathbb{R}^{100}$ and split the features among 5 parties (each holding 20 entries of $\mathbf{x}$ per case). The function we want to predict is of the form $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + \mathcal{N}(0, 0.01)$, where $\mathbf{w}$ is nonzero for half of it's entries (randomly pick half of the features to matter, and other half to be irrelevant). For now, we take each coordinate of $\mathbf{x}$ to be i.i.d. Gaussian. We compare federated gradient descent with learning rate $\eta = 1/10000$ and $T = 300000$ (iterations) against Ridge regression with $\alpha = 0.5$.

```
>>> Ridge regression
Train Error: 8.745205646168433e-05
Test Error: 0.00011756193048215462
>>> Federated gradient descent
Train 9.016817380303679e-05
Test 0.0001197005474922649
```

This is not very surprising, since the above method is functionally equivalent to linear regression with gradient descent. One natural question to ask is do we need to broadcast weights every iteration? Here, we have each party only broadcast an updated $\mathbf{w} \cdot \mathbf{x}$ inner product every k rounds.

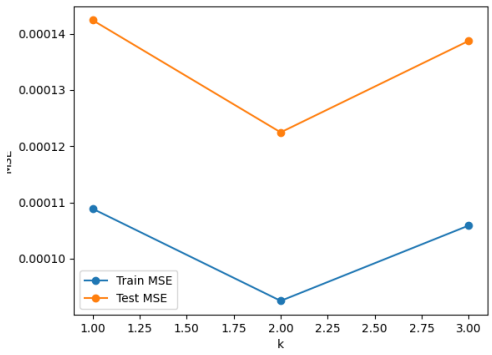

Figure 1: Graph of test and train MSE against gradient update frequency. Each party shares updated $\mathbf{w} \cdot \mathbf{x}$ every k rounds. Note that model diverges for $k \geq 4$.

Looking at this graph, we observe that updating the gradient less often seems to perform better. We note that locally at a point, the gradient does not change much, so this can be seen as some approximation of a larger learning rate. Rerunning with $\eta = 1/5000$ (doubling the learning rate), $k \geq 2$ diverges.

3.2 Gradient Inversion

Earlier, we claimed that this method allowed data to be hidden from other parties. However, this is not exactly true. Note that at each step, every party broadcasts $\mathbf{x}_i \cdot \mathbf{w}_t$, where $\mathbf{x}_i$ is the ith sample in the training set and $\mathbf{w}_t$ is their weight at time t.

Suppose there are N samples in the training set $\mathbf{x}_i \in \mathbb{R}^m$, and we record T training iterations of $\mathbf{x}_i \cdot \mathbf{w}_t$ for $i \in [N]$ and $t \in [T]$. Then we obtain the following matrix of inner products

$$A = \begin{bmatrix} \mathbf{x}_1 \cdot \mathbf{w}_1 & \cdots & \mathbf{x}_1 \cdot \mathbf{w}_T \\ \mathbf{x}_2 \cdot \mathbf{w}_1 & \cdots & \mathbf{x}_2 \cdot \mathbf{w}_T \\ \vdots & \ddots & \vdots \\ \mathbf{x}_N \cdot \mathbf{w}_1 & \cdots & \mathbf{x}_N \cdot \mathbf{w}_T \end{bmatrix} = \begin{bmatrix} \mathbf{x}_1^\intercal \\ \mathbf{x}_2^\intercal \\ \vdots \\ \mathbf{x}_N^\intercal \end{bmatrix} \begin{bmatrix} \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_T \end{bmatrix} = \mathbf{X}\mathbf{W}, \quad (4)$$

where $\mathbf{X} \in \mathbb{R}^{N \times m}$ is the data matrix and $\mathbf{W} \in \mathbb{R}^{m \times T}$ is the weight matrix (each column is from a different gradient descent iteration). This brings up the question if it is possible to recover $\mathbf{X}$ and $\mathbf{W}$, up to some linear transformation? Such a protocol would effectively leak the training data, defeating the purpose of federated gradient descent.

Pattern matching the simplest decomposition technique we know, let's apply SVD to $A = U\Sigma V^\intercal$, where $U \in \mathbb{R}^{N \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$, and $V \in \mathbb{R}^{T \times r}$. Knowing A has rank at most m, all but at most m singular values in Σ will be effectively zero, thus we can drop all columns of U and V after this point. By inspection, U looks similar to $\mathbf{X}$ and V to $\mathbf{W}$, up to some feature selection and scaling in Σ. Simply taking SVD is not sufficient to convincingly recover the features, as we show later, but it can act as a signal for how much information we have collected on the input data. *Due to time constraints, we were not able to successfully implement a complete gradient inversion attack from first principles.*

3.2.1 SVD Experiments

To investigate the effectiveness of such an attack, we restrict our view to uncorrelated features with fixing all parameters but selection of gradient descent iterations. The main question we want to answer is how does the SVD behave when varying how iterations are collected?

We randomly generate 1000 data points with $\mathbf{x} \in \mathbb{R}^{100}$ and split the features among 5 parties (each holding 20 entries of $\mathbf{x}$ per case). The function we want to predict is of the form $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + \mathcal{N}(0, 0.01)$, where $\mathbf{w}$ is nonzero for half of it's entries (randomly pick half of the features to matter, and other half to be irrelevant).

Running 100000 iterations of federated gradient descent with $\eta = 1/10000$, we obtain training error of approximately 0.0009679.

Recording the inner products $\mathbf{x}_i \cdot w_t$ every 100 iterations in A from earlier, we perform SVD and obtain singular values that look like the following

```
[2.46275299e+04 1.09897010e+03 4.37783576e+02 1.53447976e+02
```

```
6.93439793e+01 5.09262877e+01 1.08678067e+01 5.87973330e+00
2.87236217e+00 1.36803940e+00 6.79608095e-01 4.18334821e-01
1.54991701e-01 8.56768728e-02 3.38512748e-02 2.03707584e-02
6.41886637e-03 2.42541993e-03 1.21015032e-03 1.02573618e-03
1.12581855e-11 1.01822494e-11 9.73596307e-12 8.90413291e-12
7.37457279e-12 7.14802986e-12 5.74554355e-12 4.78937451e-12
4.53135626e-12 4.09240952e-12 3.95514437e-12 3.78223281e-12
3.64570819e-12 3.26467281e-12 3.02962414e-12 2.79186226e-12
2.78343760e-12 2.47092343e-12 2.39865680e-12 2.37340873e-12
2.06942979e-12 1.77723191e-12 1.77723191e-12 1.77723191e-12
1.77723191e-12 1.77723191e-12 1.77723191e-12 1.77723191e-12
1.77723191e-12 1.77723191e-12 1.77723191e-12 1.77723191e-12
1.77723191e-12 1.77723191e-12 1.77723191e-12 1.77723191e-12
...
```

Observe that only the first 20 singular values are actually significant, indicating we have some signal on the 20 columns of the data matrix $\mathbf{X}$. Varying the frequency at which we record gradients, it is clear that the number of iterations is directly correlated with dimension of the reconstructed space.

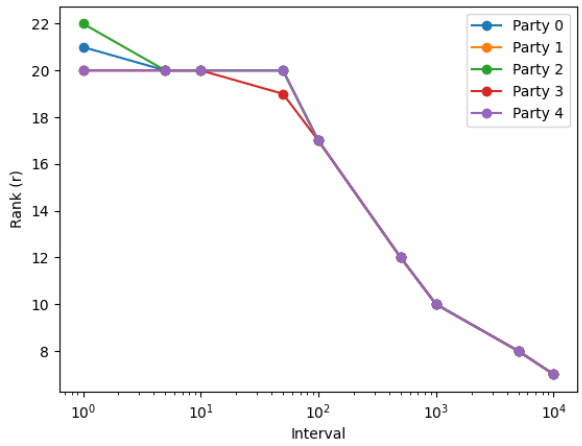

Figure 2: Plot of rank against interval for all 5 parties. Rank defined as number of singular values greater than 1e-10. Gradient is recorded every k iterations, where k = Interval.

Conjecturing that the last row of $\Sigma V^\intercal$ gives the final weights of the gradient descent iteration, we attempt to reconstruct $\mathcal{Y}$ from the SVD. Unfortunately, this gives quite terrible loss, so we need something more advanced to recover usefule features. We note that averaging over all the rows of $\Sigma V^\intercal$ does not help.

`loop`

```
    ...
    recovered_features.append(U_r)
    recovered_weights.append(np.diag(S_r) @ V_r.T)[:, -1]
Y_hat = sum(recovered_features[p] @ recovered_weights[p] for p in range(parties))
mean_squared_error(Y_hat, Y_atk)
> 2650.493480699639
```

References

[1] Mingzhe Chen, Deniz Gündüz, Kaibin Huang, Walid Saad, Mehdi Bennis, Aneta Vulgarakis Feljan, and H. Vincent Poor. Distributed learning in wireless networks: Recent progress and future challenges. *IEEE Journal on Selected Areas in Communications*, 39(12):3579–3605, 2021.

[2] Siwei Feng and Han Yu. Multi-participant multi-class vertical federated learning. *CoRR*, abs/2001.11154, 2020.

[3] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks, 2017.

[4] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated learning: Challenges, methods, and future directions. *CoRR*, abs/1908.07873, 2019.

[5] Virginia Smith, Chao-Kai Chiang, Maziar Sanjabi, and Ameet Talwalkar. Federated multi-task learning, 2018.

[6] Li Wan, Wee Keong Ng, Shuguo Han, and Vincent C. S. Lee. Privacy-preservation for gradient descent methods. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '07, page 775–783, New York, NY, USA, 2007. Association for Computing Machinery.

[7] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. Federated machine learning: Concept and applications. *CoRR*, abs/1902.04885, 2019.